

The Data & AI Challenge

Rethinking Candidate Discovery: Beyond Keywords

Team: TOPGUN

- **Nethavath Praveen** (Leader) – praveeeening@gmail.com
- **Abhinaya Nunna** – abhinayaseven@gmail.com
- **Botsa Radesh** – radeshbotsa@gmail.com
- **Sai Sameer Killamsetti** – saisameerkillamsetty8@gmail.com

Executive Summary

- **Objective:** Build an intelligent candidate discovery and ranking system.
- **Core Philosophy:** Rank candidates based on semantic meaning, career trajectory, and behavioral intent, rather than basic keyword frequency.
- **Performance:** Process 100,000+ candidates in under 35 seconds.
- **Architecture:** Fully offline, deterministic, and highly scalable.
- **Transparency:** Integrated Explainable AI (XAI) for every scored profile.

The Problem Statement

The Limitations of Traditional ATS

- **Keyword Reliance:** Traditional systems count how many times a term appears.
- **Lack of Context:** "Used Python in college" vs. "Architected a Python microservice" are scored identically.
- **Recruiter Fatigue:** Teams waste hours sifting through falsely-ranked profiles.
- **Missed Talent:** Exceptional engineers who write concise resumes get buried beneath keyword-stuffed profiles.

The Flaws of Current ATS Systems

"Keyword Stuffers" vs. True Experts

- **The Stuffer:** Pastes the entire job description in white font or tiny text. Scores 100% on legacy ATS.
- **The Expert:** Focuses on impact, system design, and leadership. Often omits basic technology tags they mastered a decade ago.
- **The Consequence:** A fundamental misalignment between the tool's output and the recruiter's actual need.

Our Solution Overview

- **Semantic Understanding:** We group skills by meaning, not string matches.
- **Career Shape Analysis:** We look at the trajectory of titles over time.
- **Intent Scoring:** We measure how actively the candidate is looking.
- **Anti-Gaming Integrity:** We statistically detect and penalize artificial inflation.
- **The Result:** A rank order that accurately mimics an expert human recruiter.

The 4 Pillars of Intelligent Ranking

Our engine evaluates candidates across four heavily weighted axes:

- ① **Semantic Competency (40%):** Technical fit across clustered concepts.
- ② **Career Trajectory (30%):** Pace of promotion and company pedigree.
- ③ **Behavioral Intent (20%):** Engagement signals and readiness to move.
- ④ **Location Intelligence (10%):** Geographical alignment with the role.

Pillar 1: Semantic Competency (40%)

Moving Beyond Exact String Matches

- **The Goal:** Recognize category expertise.
- **Mechanism:** If a job requires "Vector Databases," candidates get credit if they list Pinecone, Weaviate, FAISS, or ChromaDB.
- **Score Calculation:** Proportional to the number of critical semantic clusters a candidate possesses.
- **Bonus Allocation:** Additional weighting for full-stack capability across multiple disjoint clusters.

Skill Clusters in Practice

Cluster Category	Accepted Technologies
Core Programming	Python, Go, C++, Java
Vector Databases	Pinecone, Weaviate, Qdrant, Milvus, FAISS
Retrieval & Ranking	BM25, Cross-encoders, NDCG, Hybrid Search
LLM Engineering	Fine-tuning, LoRA, LangChain, HuggingFace
Infrastructure	Docker, Kubernetes, FastAPI, AWS, GCP

Pillar 2: Career Trajectory (30%)

Promotion Velocity and Experience Bands

- **Experience Banding:** Evaluates if the total years of experience fit the "Ideal," "Acceptable," or "Outside" parameters for the role.
- **Promotion Velocity:** Rewards candidates showing rapid progression (e.g., multiple "Senior" or "Lead" titles in a short span).
- **Quality over Quantity:** Focuses on what the candidate has achieved during their tenure, rather than purely counting months.

Dynamic Career Sequence Alignment

Applying Bioinformatics to Resumes

- **Algorithm:** Adapts the Needleman-Wunsch sequence alignment algorithm.
- **Ideal Sequence:** [Junior → Mid → Senior → Lead → Principal]
- **Execution:** Aligns a candidate's actual job titles against the ideal progression.
- **Scoring:**
 - Match Reward (+1.0)
 - Gap Penalty (-0.5 for career stagnation)
 - Mismatch Penalty (for erratic jumps)

Corporate Pedigree via PageRank

- **The Concept:** Not all experience is equal. Moving from highly-selective companies indicates a strong pedigree.
- **Implementation:** We build a directed transition graph ($Company_A \rightarrow Company_B$) across the entire candidate pool.
- **PageRank Execution:** Running a localized PageRank determines the "authority" of organizations dynamically, without hardcoding lists.
- **Result:** Candidates coming from high-authority nodes receive a strategic score boost.

Pillar 3: Behavioral Intent (20%)

Measuring Engagement

- **Recruiter Response Rate:** Candidates who historically reply to outreach rank higher.
- **Open-to-Work Status:** Active job seekers get a meaningful visibility boost.
- **Platform Activity:** Recent logins and updates signal genuine interest.
- **Notice Period:** Shorter notice periods are rewarded; heavily extended ones receive slight penalties depending on the role urgency.

Pillar 4: Location Intelligence (10%)

Geographical Alignment

- **Configurable Zones:** Roles define preferred and acceptable cities.
- **Preferred Cities (e.g., Pune, Delhi NCR):** Grants full location score (1.0).
- **Acceptable Cities (e.g., Bangalore, Mumbai):** Grants moderate score (0.7).
- **Remote / Unlisted:** Grants baseline score (0.4) to avoid entirely eliminating remote-willing candidates.

System Integrity & Anti-Gaming

Defeating the Keyword Stuffers

- Our system automatically filters deceptive profiles into a "honeypot" quarantine.
- **Impossible Claims:** Flags "Expert" proficiency attached to 0 months of usage.
- **Timeline Inconsistencies:** Flags claims of 10+ years of experience that do not map to the actual career timeline provided.
- Protects the integrity of the recruiter's shortlist.

Shannon Entropy Anomaly Detection

Statistical Profiling

- **The Theory:** Natural human language has a predictable distribution of vocabulary.
- **The Math:** We calculate the Shannon Entropy of the resume text.

$$H(X) = - \sum P(x_i) \log_2 P(x_i)$$

- **The Trigger:** Profiles whose vocabulary entropy deviates by more than 2σ from the population mean are automatically flagged.
- **The Result:** Successfully catches machine-generated, keyword-stuffed, and white-fonted resumes.

- **Two Primary Layers:**
 - ① Parallelized Python Engine (Offline CLI)
 - ② Interactive Recruiter Sandbox (Frontend UI)
- **Ingestion:** $O(N)$ streaming scanner restricts memory footprint under 200MB.
- **Pre-Scan Phase:** Calculates global TF-IDF weights and entropy baselines.
- **Execution:** Multiprocessing pool shards candidates across CPU cores.

The Parallelized Scoring Engine

- **Speed:** Processes batches of candidates concurrently across all available cores.
- **Benchmark:** 100,000+ candidate profiles ranked in **under 35 seconds** on a standard CPU.
- **Efficiency:** Achieves execution well within the 5-minute hackathon limit.
- **Deterministic Sort:** Ranks by $(-round(score, 4), candidate_id)$ guaranteeing identical results on every rerun.

Explainable AI (XAI)

Generating Deterministic Rationale

- **No Black Boxes:** Every shortlist placement must be justifiable.
- **Context-Aware Generation:** The engine writes human-readable rationale based on actual matched parameters.
- **Example Output:** *"Exceptional technical fit, covering 4/5 core semantic clusters. 6.5 years of steady progression. Background includes Tier-1 product companies."*
- Builds trust between the algorithm and the recruiter.

Fully Offline Operation

Meeting Strict Privacy and Latency Constraints

- **Zero API Calls:** The entire pipeline runs using Python's standard library.
- **No OpenAI/Cloud Dependency:** Eliminates network latency, API rate limits, and token costs.
- **Security:** Enterprise-ready approach where candidate PII never leaves the local machine.
- Complies 100% with sandbox environment restrictions.

Configurable Job Descriptions

- **Adaptability:** No code changes required to hire for a different role.
- **JSON Driven:** All role requirements exist in `data/jd_rules.json`.
- **Customization:** Recruiters can easily modify:
 - Target Role Title
 - Experience Minimums and Maximums
 - Semantic Skill Clusters
 - Preferred Locations and Company Tiers

The Recruiter Dashboard UI

Zero Setup, Maximum Insight

- **The Technology:** A pure HTML/JS single-file application. No Node.js, no build steps.
- **Accessibility:** Runs directly in any modern browser immediately after the CLI engine finishes.
- **Purpose:** Translates the raw CSV output into an interactive, visual sandbox for recruiters.

Dashboard Features in Detail

- **CSV Upload:** Drag-and-drop the generated `team_submission.csv`.
- **Candidate Cards:** Visual display of rank, score, and the AI rationale.
- **Click-to-Expand:** Opens a detailed scoring breakdown with individual progress bars for Skills, Experience, and Intent.
- **Export Capabilities:** One-click re-export of filtered/sorted results.

Active Learning Loop (RLRF)

Continuous Improvement

- **Recruiter Feedback:** Dashboard allows recruiters to "Invite" or "Decline" candidates.
- **Weight Adjustment:** The system uses a single-layer perceptron training step to update category weights based on user preference.
- **Adaptation:** Over time, the local weights shift to match the specific hiring manager's implicit preferences.

Performance Metrics & Constraints

Metric	Value
Dataset size	100,000+ candidate profiles
Processing time	~34 seconds
Memory usage	< 500 MB RAM
External API calls	Zero
Dependencies	Pure Python stdlib (Core Engine)
Hackathon Time Limit	5 minutes (We are 8× faster)

Conclusion & Impact

- **For Recruiters:** Drastically reduces time-to-shortlist from days to 35 seconds.
- **For Candidates:** Rewards actual career growth, system design skills, and intent—not just resume hacking.
- **For Redrob:** A highly scalable, production-ready heuristic engine adaptable to any JD via a simple configuration.

Thank you! Check out our GitHub Repo.